

Problem Statement

Develop a basic application that integrates a frontend developed using Electron and Blockly, a backend Python script with tkinter functionality, MySQL database interactions, and post-execution communication between the frontend and backend via websockets.

Note: All paths in the file should be dynamic (not hard-coded)

Requirements

1. Frontend (Electron + Blockly)

- Set up a Blockly workspace in an Electron app.
- Include a "Compile" button to generate a CSV file from the Blockly logic. The file must be saved to a predefined local directory.
- Include a "Run" button that:
 - Launches the Python script.
 - Instructs the Python script to process the previously compiled CSV.

2. Backend (Python with tkinter and MySQL)

- Create a Python script that:
 - Reads the compiled CSV file from the predefined directory.
 - Executes logic blocks (e.g., if-else, for loop) described in the CSV.
 - Fetches data from a MySQL database if required by the logic.
 - Displays the output in a tkinter window.
 - After execution, establishes a websocket connection with the Electron frontend to send the status of each block execution.

3. Database (MySQL)

- Use the provided MySQL database.

4. Frontend-Backend Communication

- Websockets connection is established only after the Python script finishes processing the CSV file.
- Use the websocket to notify the frontend of:
 - Execution status of each block in the CSV.
 - Update block colors in Blockly (green for success, red for failure).

Tasks

Task 1: Frontend

1. Create a Blockly workspace with the following blocks:
 - if-else
 - for loop
 - fetch from database
 - Basic math operations (add, subtract, multiply, divide).
2. Add a "Compile" button that:
 - Generates a CSV file representing the current Blockly workspace.
 - Saves the file to a predefined directory (e.g., ./csv_output/).
3. Add a "Run" button that:
 - Executes the Python script to process the CSV file.

Task 2: Backend

1. Create a Python script that:
 - Reads the CSV file from the predefined directory.
 - Parses the CSV and executes the commands row by row:
 - For fetch from database blocks, query the MySQL database and use the results in the logic.
 - Handle logic like if-else and for loop as described in the CSV.
 - Displays the output of the execution in a tkinter window.
2. After completing the execution:
 - Establish a websocket connection with the frontend.
 - Send execution statuses for each block (success/failure).
3. Ensure error handling for blocks that fail during execution.

Task 3: MySQL Database

- Use the predefined MySQL database.
- Ensure the Python script connects to the database to fetch data when required by the logic in the CSV file.

Task 4: Frontend-Backend Communication

1. Establish a websocket connection between the frontend and backend only after the Python script finishes execution.
2. Use the websocket to send the following information from the backend to the frontend:

- Execution status for each block (e.g., success/failure).
3. Update the Blockly workspace to:
- Turn blocks green if successfully executed.
 - Turn blocks red if execution failed.

Deliverables

1. Fully functional Electron app with Blockly integration.
2. Python backend script with:
 - tkinter output.
 - MySQL query integration.
3. Communication between the frontend and backend via websockets (post-execution).
4. A predefined CSV directory where the Electron frontend saves compiled CSV files.
5. MySQL database setup with provided file.

-- End of Document --